High Level Design On

Containerized Marketing Service with Docker

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Bhavya Rathore	EN22CS301269
2	Bhoomika Sanodiya	EN22CS301271
3	Bhumika Chouhan	EN22CS301275
4	Devi Sanjaykumar Modi	EN23CS3T1008
5	Dollyna Sahu	EN22CS301354

Group Name: Group G2D3

Project Number: DO-26

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Prof. Shyam Patel

Academic Year: 2025-2026

Table of Contents

1. Introduction.
   - 1.1. Scope of the document.
   - 1.2. Intended Audience
   - 1.3. System overview.
2. System Design.
   - 2.1. High Level Architecture
   - 2.2. Module Design
   - 2.3. Data flow Architecture
   - 2.4. System Workflow
   - 2.5. API Catalogue
3. Architecture Diagram
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
   - 7.1. Security Aspects
   - 7.2. Performance Aspects
8. References

1. Introduction

The rapid growth of web-based applications has increased the demand for scalable, maintainable, and easily deployable backend systems. Modern software development practices emphasize modular architecture, containerization, and automation to ensure consistent performance across different environments such as development, testing, and production.

The AI Marketing Agent is a cloud-native application designed to generate intelligent marketing content using Large Language Model (LLM) capabilities via Groq API. The system is containerized using Docker and deployed on AWS EC2, with Kubernetes support for orchestration. The project demonstrates a complete DevOps workflow including containerization, image management, and cloud deployment.

1.1 Scope of the Document

This document defines the high-level architecture and design of the system. It covers system components, data flow, deployment architecture, APIs, and non-functional requirements.

Scope includes:

- System architecture design
- Docker-based containerization approach
- Cloud deployment using AWS EC2
- Kubernetes orchestration overview
- API integration (Groq AI)
- DevOps workflow explanation

The objective is to ensure that the system is environment-independent and can run consistently across all stages of the software development lifecycle.

1.2 Intended Audience

This document is intended for:

- Software Developers
- DevOps Engineers
- System Architects
- Cloud Engineers
- Academic Evaluators
- Project Review Panels

1.3 System Overview

The system is a web-based AI application built using Python and Streamlit framework. It interacts with the Groq API to generate marketing content based on user input. The application is containerized using Docker and deployed on AWS EC2 for production usage. Kubernetes is used for orchestration and scaling demonstration.

Key Highlights:

- AI-powered content generation
- Containerized deployment using Docker
- Cloud hosting on AWS EC2
- Kubernetes-based orchestration (Minikube)
- Scalable and portable architecture

2. System Design

The system follows a modular DevOps-based architecture where each component is loosely coupled and deployed using containerization principles.

2.1 Application Design

The application is designed using a simple layered architecture:

Layers:

- Presentation Layer: Streamlit UI
- Application Layer: Python backend logic
- AI Layer: Groq API integration
- Deployment Layer: Docker + AWS EC2

Key Points:

- Single container-based deployment
- Stateless architecture
- API-driven communication
- Modular design for scalability

2.2 Process Flow

The system follows a sequential request-response flow:

Steps:

- User enters input through Streamlit UI
- Input is processed by Python backend
- Backend sends request to Groq API
- AI model generates marketing response
- Response is returned to UI
- Output is displayed to user

2.3 Information Flow

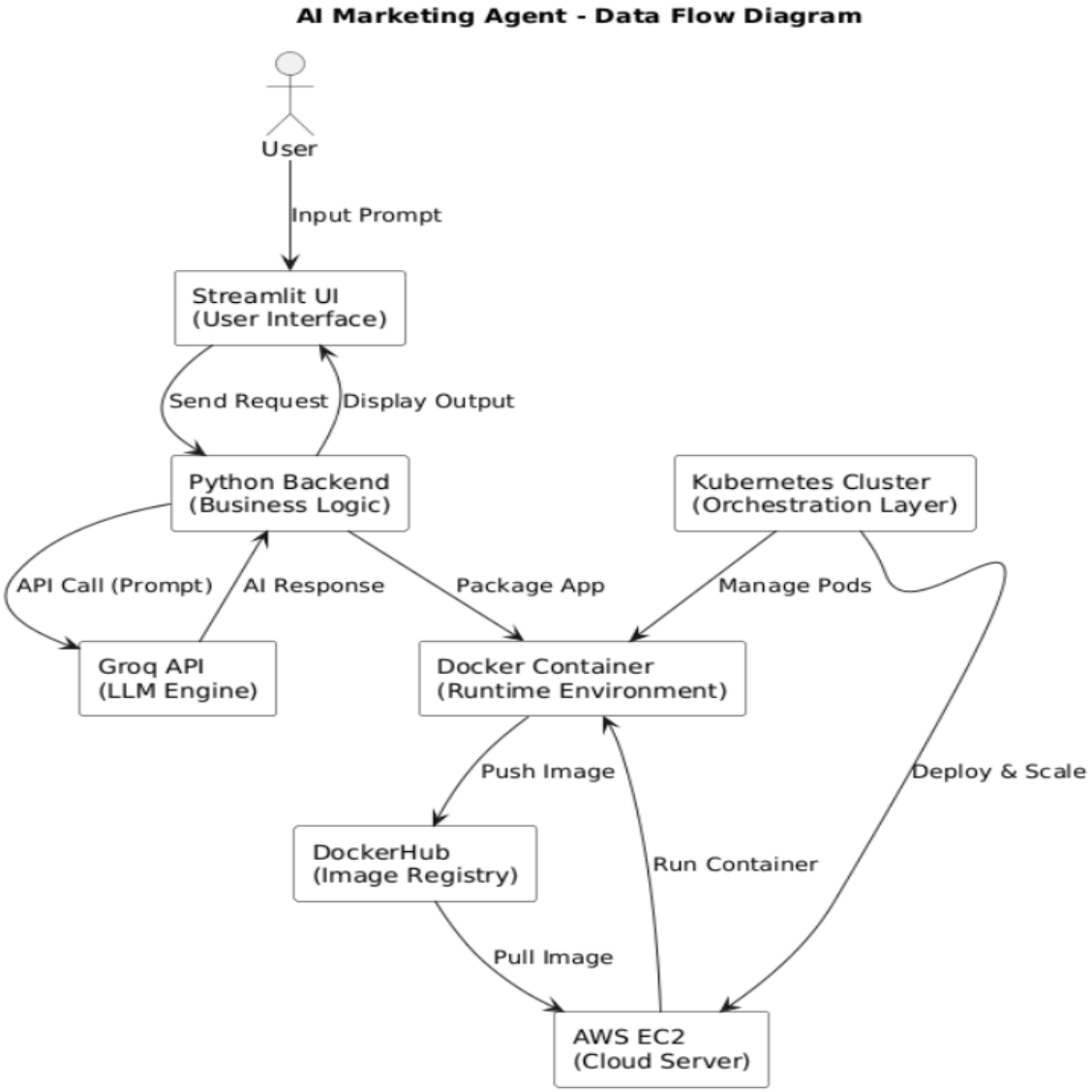

Figure 1: Data Flow Diagram

Data Flow Architecture:

User → Streamlit UI → Python Backend → Groq API → Response → UI Display

Characteristics:

- Stateless communication
- API-driven processing
- Real-time response generation
- No persistent storage required

2.4 Components Design

The system is divided into multiple logical components:

1. Frontend Component

- Streamlit-based UI
- Handles user interaction

2. Backend Component

- Python logic
- API request handling

3. AI Component

- Groq LLM API
- Generates marketing content

4. Containerization Layer

- Docker used for packaging application
- Ensures portability across environments

5. Cloud Layer

- AWS EC2 used for hosting
- Public accessibility enabled via IP

6. Orchestration Layer

- Kubernetes (Minikube)
- Handles deployment simulation and scaling

2.5 Key Design Considerations

Architecture Decisions:

- Containerization for environment consistency
- Cloud deployment for scalability
- Stateless design for performance
- API-based modular structure

Technical Considerations:

- Secure API key handling via environment variables
- Lightweight container image design
- Fast AI response using Groq API
- Kubernetes readiness for production scaling

2.5 API Catalogue

API Name	Purpose
Groq Chat API	Generates AI-based marketing content
Streamlit Interface API	Handles UI interactions
Docker API	Container execution
AWS EC2 API	Cloud hosting and deployment

3. Architecture Diagram

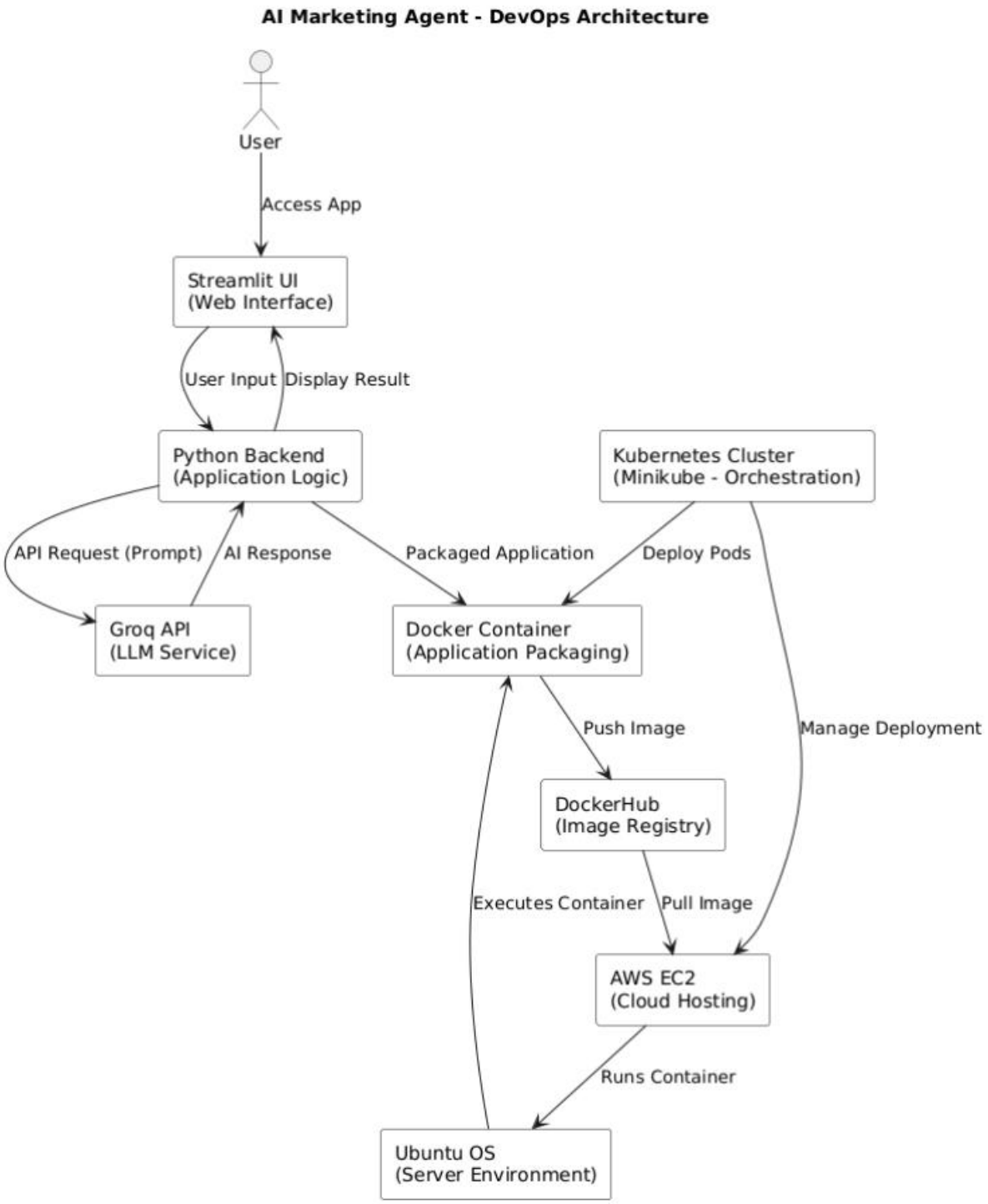

Figure 1: Architecture Diagram

4. Interfaces

The system uses multiple interfaces to enable communication between different layers of the application, external services, and deployment platforms. These interfaces ensure smooth interaction between user requests, backend processing, AI services, and cloud infrastructure.

4.1 User Interface (UI)

The User Interface is the front-facing component of the system developed using Streamlit. It provides an interactive web-based platform for users to input marketing prompts and view generated AI responses.

Features:

- Simple and interactive web UI
- Text input box for user prompts
- Real-time display of AI-generated marketing content
- Lightweight browser-based access
- No installation required for end users

Functionality:

- Accepts user input (marketing query or prompt)
- Sends request to backend application
- Displays response from Groq API in formatted output

4.2 System Interfaces

System interfaces define communication between internal components and external services used in the application.

Key System Interfaces:

1. Groq API Interface

- Used for AI-based text generation
- Accepts prompt from backend
- Returns generated marketing content
- REST-based HTTP communication

2. Docker Interface (Docker Engine)

- Used to build and run application containers
- Provides isolated runtime environment
- Ensures portability across systems

3. AWS EC2 Interface

- Provides cloud compute environment
- Hosts Docker container in production
- Allows public access via IP address
- Managed via SSH and AWS console

4. Kubernetes Interface (kubectl / Minikube)

- Used for container orchestration
- Manages pods and deployments
- Provides scaling and self-healing features
- Simulates production-grade deployment environment

4.3 Communication Interfaces

Communication interfaces define how data is transferred between different components of the system.

Types of Communication:

1. REST API Communication

- Used between backend and Groq API
- Uses HTTP GET/POST requests
- JSON format for request and response

2. HTTP/HTTPS Protocol

- Used for web access via Streamlit UI
- Ensures secure communication over network

3. CLI-Based Communication

- Docker CLI for container management
- Kubernetes CLI (kubectl) for cluster operations
- AWS CLI / SSH for server access

User Interface & Screenshot

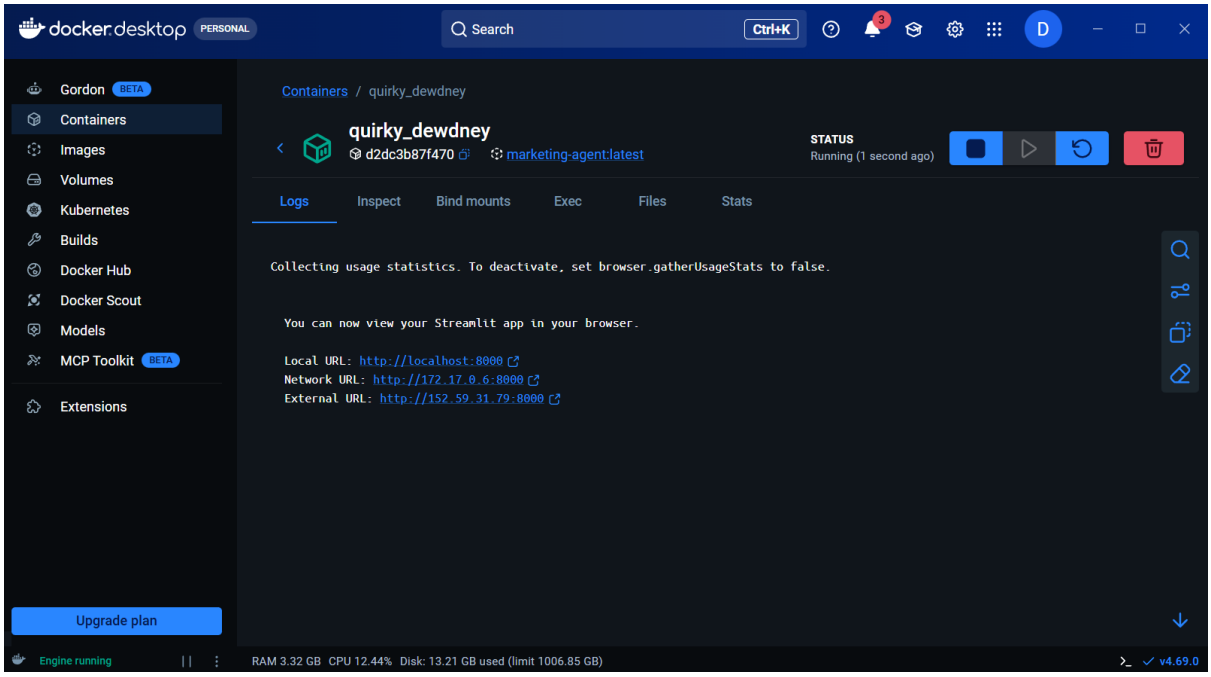

Figure 3: Screenshot of Docker Interface

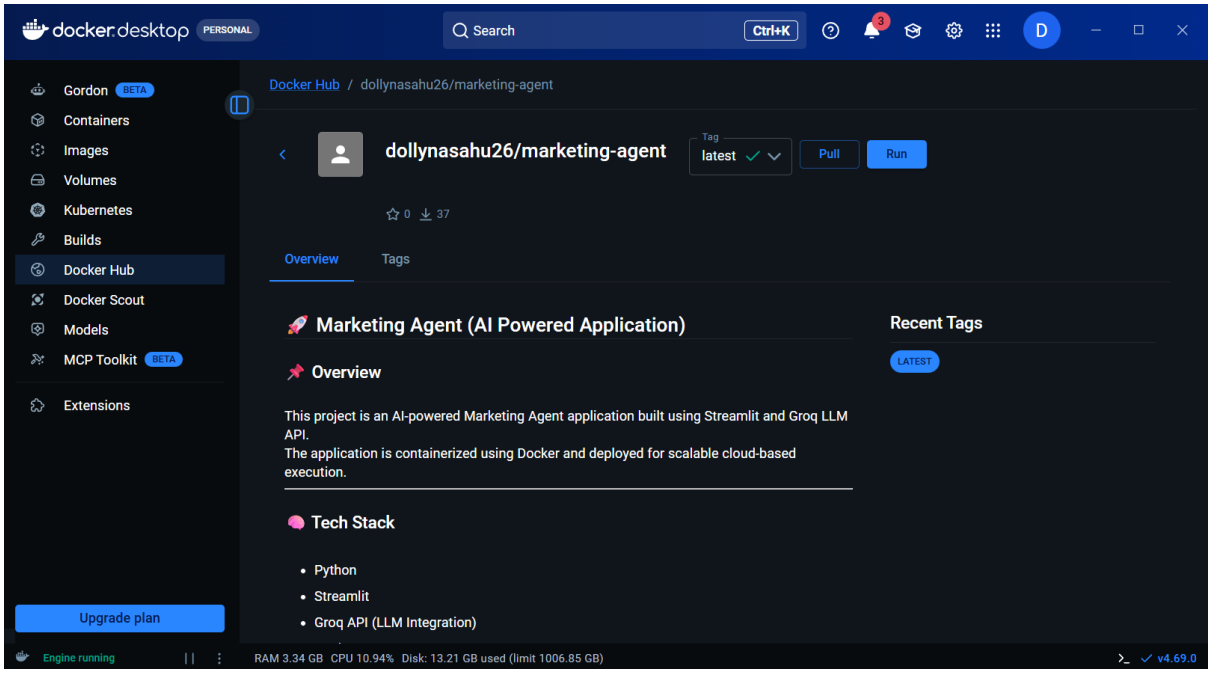

Figure 3: Screenshot of DockerHub Interface

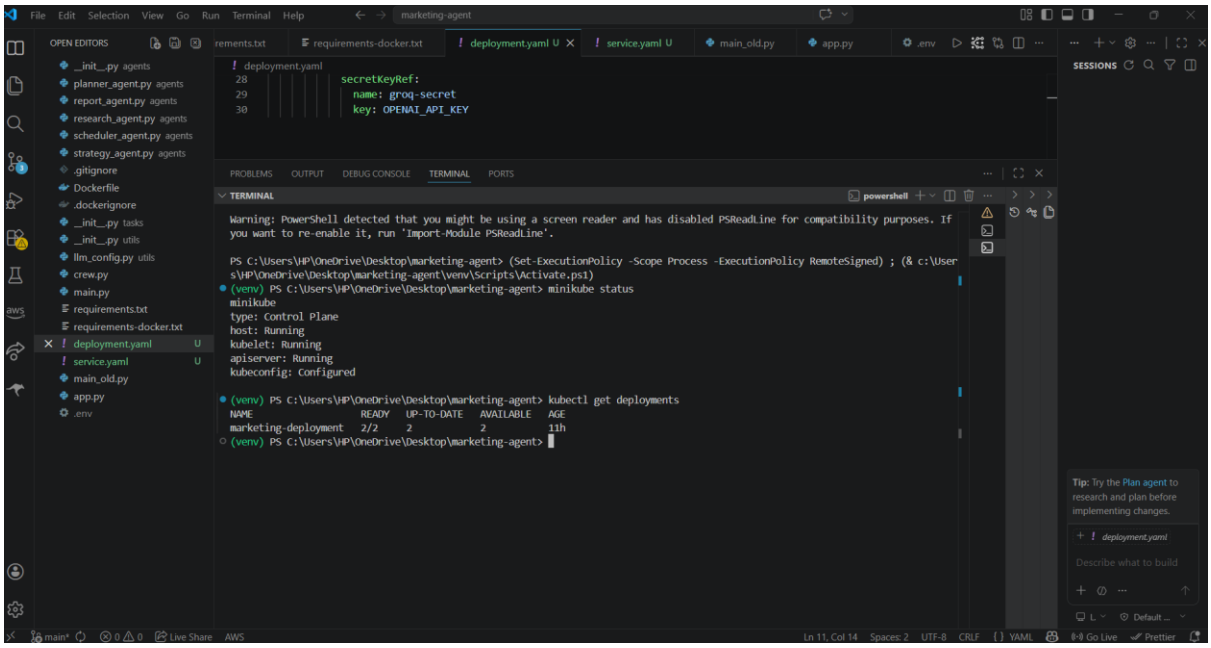

Figure 3: Screenshot of Minikube(Kubernetes Integration)

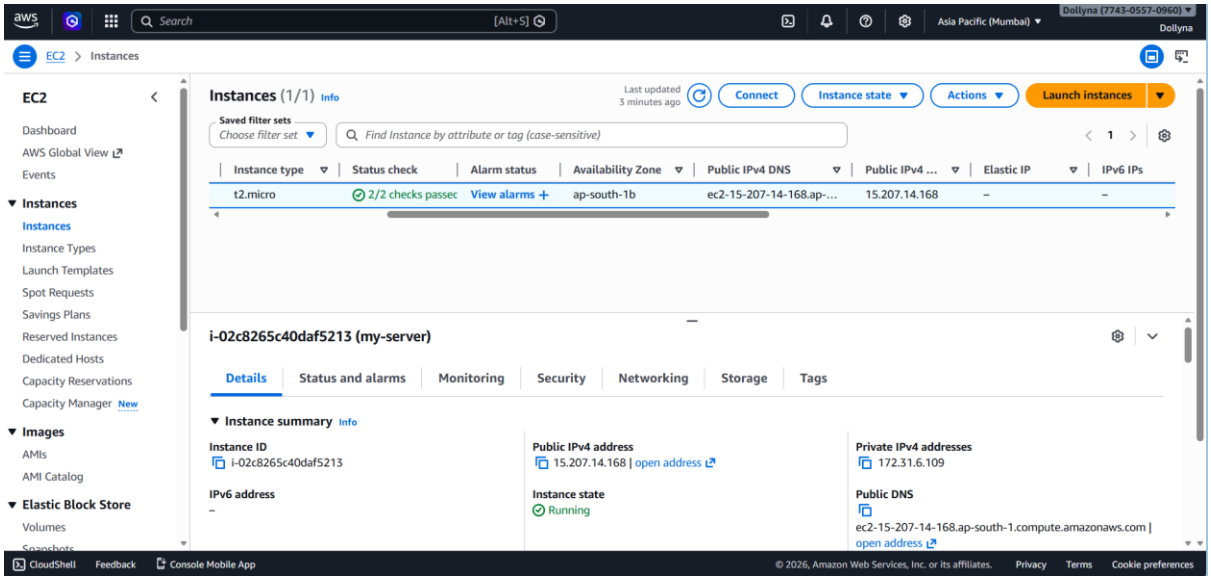

Figure 3: Screenshot of EC2 Instance Interface

5. State and Session Management

The AI Marketing Agent follows a stateless architecture, meaning the system does not store persistent user data or maintain long-term sessions on the server. Each request is processed independently, ensuring scalability, simplicity, and cloud compatibility.

Since the application is deployed using Docker and hosted on AWS EC2, session handling is minimized to improve performance and maintain container portability.

5.1 State Management

State management refers to how the application handles data during execution.

Key Characteristics:

- The application is stateless at the backend level
- No permanent user data is stored on server
- Each API request is treated as a new transaction
- Temporary state exists only during request execution

How State is Handled:

- User input is received via Streamlit UI
- Input is processed in memory (Python runtime)
- Request is sent to Groq API
- Response is generated and immediately returned to UI
- No historical data is retained after response generation

Benefits of Stateless Design:

- Easy to scale horizontally in cloud environments
- No dependency on server memory
- Simplifies Docker container deployment
- Improves fault tolerance

5.2 Session Management

Session management defines how user interactions are tracked during runtime.

- Streamlit provides lightweight session handling

- Sessions are maintained only at UI level (frontend)
- No backend session persistence is implemented
- Each session is independent and temporary

Streamlit Session Behavior:

- Maintains UI state during active browser session
- Stores temporary variables in memory
- Resets when browser is refreshed or closed

6. Caching

Caching refers to storing frequently accessed data temporarily to improve system performance and reduce response time. In the AI Marketing Agent system, caching is intentionally minimal due to the real-time nature of AI-generated responses.

Caching Strategy

The system does not implement persistent caching mechanisms such as Redis or Memcached. Instead, it relies on direct API calls to ensure real-time accuracy of responses.

Key Points:

- No external caching layer is used
- Each user request is processed independently
- Responses are generated in real-time using Groq API
- Streamlit does not cache backend responses permanently

7. Non-Functional Requirements

Non-functional requirements define system quality attributes such as security, performance, scalability, and reliability. These requirements ensure that the AI Marketing Agent performs efficiently in a cloud environment.

7.1 Security Aspects

Security is a critical part of the system due to API usage and cloud deployment.

1. API Key Protection

- Groq API keys are stored as environment variables
- No hardcoding of sensitive credentials in code
- Docker runtime injects environment variables securely

2. AWS Security Groups

- EC2 instance access controlled via security groups
- Only required ports (e.g., 8501) are exposed
- SSH access restricted using key pairs

3. Container Isolation

- Docker provides isolated runtime environment
- Prevents direct system-level access

4. Access Control

- EC2 access secured using PEM key authentication
- No password-based login enabled

Security Benefits:

- Reduced risk of credential leakage
- Secure cloud deployment
- Isolated application runtime
- Controlled network access

7.2 Performance Aspects

Performance requirements ensure fast response time and efficient resource utilization.

Performance Features:

1. Lightweight Containerization

- Docker ensures minimal overhead
- Fast startup time of application

2. Efficient AI Processing

- Groq API provides low-latency LLM inference
- Faster response compared to traditional LLM APIs

3. Cloud Scalability

- AWS EC2 provides scalable compute resources
- Instance type (t2.micro) supports lightweight workloads

4. Stateless Architecture

- No session persistence reduces memory usage
- Enables horizontal scaling

5. Streamlit UI Efficiency

- Lightweight frontend framework
- Minimal rendering delay

8. References

[1] Docker Inc., *Docker Documentation*, 2026. [Online]. Available: https://docs.docker.com. [Accessed: 20-Apr-2026].

[2] The Kubernetes Authors, *Kubernetes Documentation*, 2026. [Online]. Available: https://kubernetes.io/docs. [Accessed: 20-Apr-2026].

[3] Amazon Web Services, *Amazon EC2 Documentation*, 2026. [Online]. Available: https://docs.aws.amazon.com/ec2. [Accessed: 20-Apr-2026].

[4] Groq Inc., *Groq API Console Documentation*, 2026. [Online]. Available: https://console.groq.com. [Accessed: 20-Apr-2026].

[5] Streamlit Inc., *Streamlit Documentation*, 2026. [Online]. Available: https://docs.streamlit.io. [Accessed: 20-Apr-2026].